Estructura de Datos
Semana 03: Diagramación de Punteros y Estados de Memoria (Stack, Heap y GC)

Grupo 02:
Darwin Campoverde
Javier Guarnizo
Ricardo Ochoa
Diyer Torres
Mateo Yanangomez

Carrera de Computación – Universidad Nacional de Loja

18 de octubre de 2026

1. Guía de Ejercicios Avanzados de Memoria en JavaScript

Caso de Estudio 1: Shared Lexical Context Leak

Considere el siguiente código en JavaScript ejecutado en el motor V8:

```
let ejecutorGlobal = null;

function crearProcesadorResiduos() {
  // Asignacion de 10 MB en el Heap
  let datosMasivos = new Array(10 * 1024 * 1024).fill("DATA_CRITICA");
  let copiaAnterior = ejecutorGlobal;

  // Funcion interna 1: NUNCA se invoca directamente, pero utiliza datosMasivos
  let funcionRetenida = function () {
    if (datosMasivos) {
      console.log("Referencia activa a datos masivos");
    }
  };

  // Funcion interna 2: Sobrevive asignandose a la variable global
  ejecutorGlobal = function () {
    console.log("Ejecutando proceso liviano...");
    if (copiaAnterior) {
      copiaAnterior();
    }
  };
}
```

```
23
24 setInterval(() => {
25   crearProcesadorResiduos();
26   console.log(‘Heap usado: ${(process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2)} MB‘);
27 }, 1000);
```

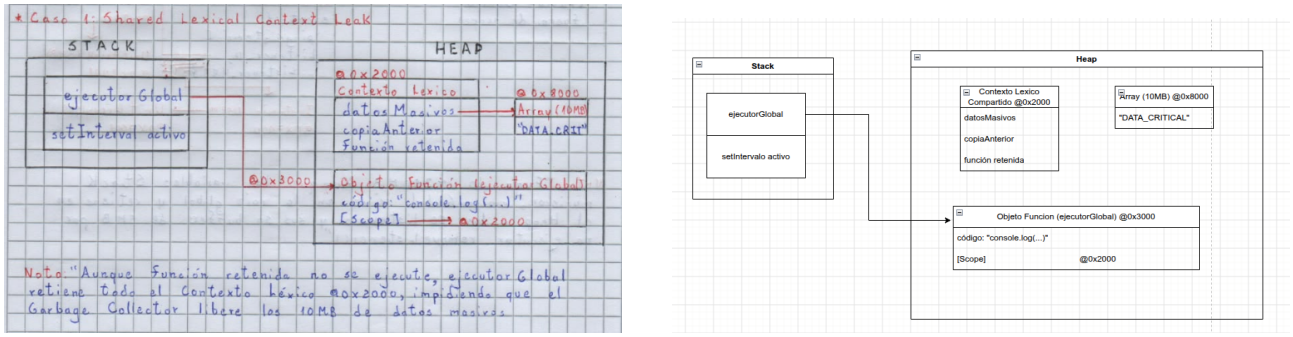
Analice la evolución del Stack y Heap y justifique la causa del aumento paulatino de memoria.

Demostración.
Análisis Teórico y Mecanismo del V8 Engine:

a) **Contexto Léxico Compartido:** Al crearse las dos funciones internas dentro de `crearProcesadorResiduos()`, el motor V8 asigna un único *Shared Lexical Context* que contiene todas las variables del ámbito padre capturadas por alguna de las funciones internas.

b) **Retención Involuntaria:** Aunque `funcionRetenida` no se ejecute jamás, el mero hecho de que referencie a `datosMasivos` fuerza a que dicha variable viva en el Contexto Léxico. Puesto que `ejecutorGlobal` sobrevive globalmente y mantiene su propiedad `[[Scope]]` apuntando a dicho contexto, los 10 MB de `datosMasivos` no pueden ser liberados por el Garbage Collector.

Diagramación del Estado de Memoria:



(a) Diagrama dibujado a mano. **(b)** Diagrama vectorial digitalizado.

Figura 1: Caso 1: Retención de memoria por Shared Lexical Context en V8.

□

Caso de Estudio 2: Grafo Circular y Referencias Débiles

Analice la estructura de datos orientada a grafos y evalúe la recolección de basura ante la invocación de `gestorCache.delete(idRaiz)`:

```
class NodoGrafo {
  constructor(id) {
    this.id = id;
    this.vecinos = [];
  }
  conectarCon(nodo) {
    this.vecinos.push(nodo);
  }
}

function construirGrafoCiclico() {
  let nodoA = new NodoGrafo("A");
  let nodoB = new NodoGrafo("B");
  let nodoC = new NodoGrafo("C");

  // Conexion circular bidireccional
  nodoA.conectarCon(nodoB);
  nodoB.conectarCon(nodoC);
  nodoC.conectarCon(nodoA);

  // Registro en un gestor centralizado
  gestorCache.set(nodoA.id, nodoA);
  return nodoA.id;
}

const gestorCache = new Map(); // Map o WeakMap?
let idRaiz = construirGrafoCiclico();
// gestorCache.delete(idRaiz);
```

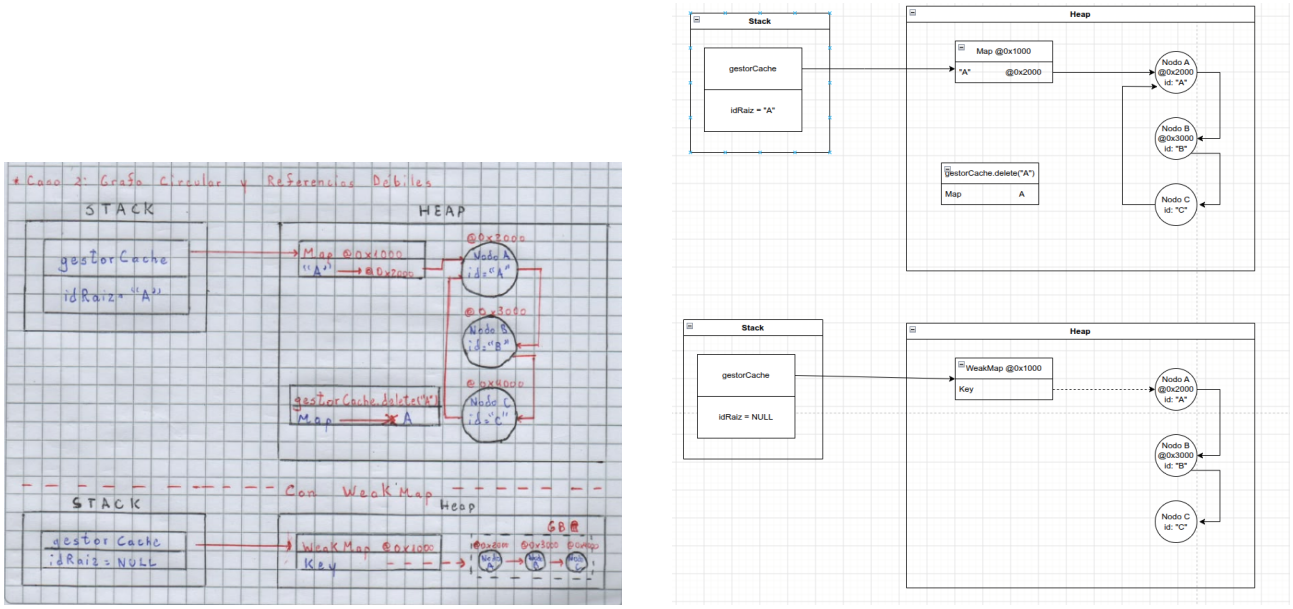
Demostración.

Análisis del Algoritmo Mark-and-Sweep:

a) **Comportamiento con `Map`:** Las claves y valores en un `Map` convencional son referencias fuertes. Si los objetos se mantienen dentro del diccionario o con referencias circulares alcanzables, el algoritmo *Mark-and-Sweep* no los marca como inalcanzables.

b) **Optimización con `WeakMap`:** Al emplear `WeakMap` asignando como clave la referencia directa al objeto `nodoA`, la relación de la colección es débil. Al perderse la referencia fuerte desde la pila (`Stack`), el ciclo $A \to B \to C \to A$ queda completamente aislado de las raíces globales (*GC Roots*), logrando que todo el grafo sea liberado.

Diagramación del Estado de Memoria:



(a) Diagrama dibujado a mano. **(b)** Diagrama vectorial digitalizado.

Figura 2: Caso 2: Grafo Cíclico, algoritmos de GC y referencias débiles.

□

Caso de Estudio 3: Event Loop Queue & Retención por Callbacks

Determine la causa del consumo masivo de Heap en la simulación de tráfico de usuarios con eventos globales:

```
const EventEmitter = require('events');
const emisorGlobal = new EventEmitter();

class GestorSesionUsuario {
  constructor(idUsuario) {
    this.idUsuario = idUsuario;
    // Reserva de 5 MB de datos por sesion
    this.bufferSesion = new Uint8Array(5 * 1024 * 1024);

    // Suscripcion al emisor global
    this.oyenteEvento = () => {
      this.actualizarActividad();
    };

    emisorGlobal.on('pingSistema', this.oyenteEvento);
  }

  actualizarActividad() {
    console.log(`Sesion ${this.idUsuario} activa.`);
  }

  cerrarSesion() {
    console.log(`Cerrando sesion de ${this.idUsuario}`);
    // NOTA: El desarrollador olvido desregistrar el evento en emisorGlobal
  }
}

function simularTrafico() {
  for (let i = 0; i < 50; i++) {
    let usuario = new GestorSesionUsuario(`USR_${i}`);
    usuario.cerrarSesion();
  }
}

simularTrafico();
emisorGlobal.emit('pingSistema');
```

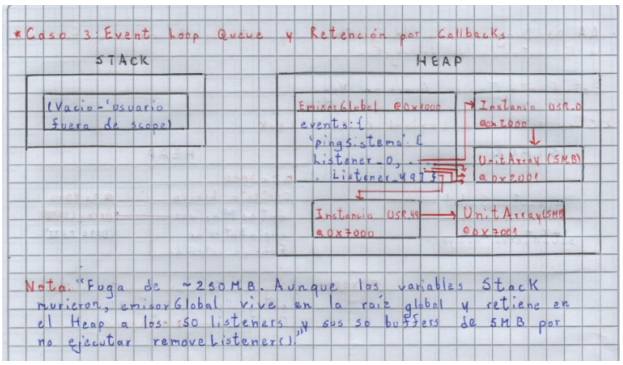
Demostración.

Análisis de Fuga por Listener Colgante (Dangling Listener):

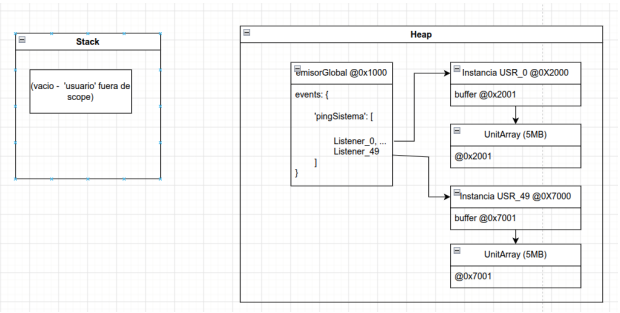
a) **Alcanzabilidad desde GC Root:** La variable `emisorGlobal` permanece viva en el ámbito global. Cada iteración suscribe una función flecha al arreglo `_events['pingSistema']`.

b) **Retención Causal:** La función flecha captura la referencia a la instancia `this` (GestorSesionUsuario). Dado que el desarrollador no ejecutó `removeListener()` dentro de `cerrarSesion()`, el motor V8 retiene las 50 instancias con sus respectivos buffers de 5 MB, sumando una fuga fija de ~ 250 MB en el Heap.

Diagramación del Estado de Memoria:



(a) Diagrama dibujado a mano.



(b) Diagrama vectorial digitalizado.

Figura 3: Caso 3: Retención por callbacks desatendidos en el Event Loop.

Solución y Mitigación: Desregistrar el listener explícitamente en el método de cierre:

```
cerrarSesion() {
  console.log(‘Cerrando sesion de ${this.idUsuario}‘);
  emisorGlobal.removeListener(’pingSistema’, this.oyenteEvento);
}
```

□